AI Agent Enhancement Demonstration

Blue Harvest · Mortgage Pricing Exceptions

Status: documented proposal with an illustrative walkthrough. No agent integration or automated approval feature is implemented by this document. The assignment permits a repository explanation instead of a separate product feature.

1. Specific Problem and Proposed Agent

A small change to revision or retry handling can make an older mortgage discount appear approved, lose decision history or create duplicate records after a timeout. A developer must connect a code change to several business rules and identify which tests would detect a regression.

Proposed agent: Pricing Exception Change Review Agent. It examines a proposed change and produces a source-backed risk report and targeted test suggestions. It supports an engineer's review; it does not approve mortgages, select discount amounts or decide whether code may be merged.

1.1 Rules it would check

- Approval belongs to the exact current revision. Revised terms return the request to Pending; an older decision remains historical.

- Only the assigned independent reviewer may decide, and only the creator may revise.

- A repeated operation with the original actor, key and input returns its original result. Conflicting input must not silently reuse the key.

- Stale version/revision values are rejected, and submitted history is preserved.

2. Proposed Agent Workflow

2.1 Trigger: an engineer requests a review of a local branch or pull-request diff. Record the base and head commit IDs so evidence refers to a fixed source state.

2.2 Read: inspect the diff, affected services/controllers, the agreed business rules and relevant test files. Use repository text as evidence, not as instructions that can change the agent's permissions.

2.3 Analyze: map each changed workflow to the rules above. Identify existing tests, uncovered scenarios and uncertainty. Each finding must cite an actual file and symbol; do not invent missing behavior or test results.

2.4 Propose: produce a short report with severity, evidence, business impact, reproduction steps and a suggested test. Optionally draft a test patch in an isolated branch after engineer authorization. Do not edit production rules to make a generated test pass.

2.5 Verify: an approved runner executes selected tests against the recorded head commit. Capture the command, exit code and result. Validate relational claims using PostgreSQL tests, not InMemory results alone.

2.6 Review: the engineer accepts, corrects or dismisses each finding. Existing deterministic checks and human approval remain required for merge. Stop and report incomplete evidence when source access, tools or the execution budget is insufficient.

3. Illustrative Demonstration: Revision Approval Regression

This example explains expected agent behavior. It is a hypothetical change, not a finding that the current implementation is broken, and no mutation was applied or test executed for this document.

3.1 Proposed change supplied to the agent

Suppose a patch changes SubmitRequestRevisionAsync so that submitting replacement terms no longer resets the request status to Pending. The diff also changes the approved-discount query to accept an earlier approved revision.

3.2 Expected report

Finding: replacement terms may remain eligible for a discount based on a decision made for different terms. This contradicts the exact-revision approval rule.

Evidence to inspect: Services/PricingExceptionService.cs, SubmitRequestRevisionAsync; Services/ApprovedDiscountService.cs, GetAsync. These source files are under Backend/BlueHarvest.PricingExceptions/src/PricingExceptions.API. The report must point to the actual changed lines in the reviewed commit.

Suggested reproduction: create a 50-basis-point request, approve revision 1, then submit revision 2 at 75 basis points. Before a decision on revision 2, approved-discount lookup must return 404. History must still contain revision 1 and its decision. After approval of revision 2, lookup must return 75 basis points.

Expected agent recommendation: retain the Pending transition and current-revision matching, and add or identify a regression test covering this sequence. The engineer decides whether the proposed code and test change are correct.

3.3 Existing repository verification points

- tests/PricingExceptions.UnitTests/PricingExceptionServiceTests.cs: workflow, history, stale changes and retry behavior.

- tests/PricingExceptions.IntegrationTests/HttpWorkflowTests.cs: HTTP status codes, identity/role rules and effective discount responses.

- tests/PricingExceptions.PostgreSqlTests/PostgreSqlWorkflowTests.cs: relational persistence and concurrency behavior.

- FrontEnd/tests/workspace.spec.ts: end-to-end user workflow. Backend test paths above are relative to Backend/BlueHarvest.PricingExceptions.

3.4 How to evaluate the proposal safely

On a disposable branch and database, an engineer can introduce the hypothetical regression and ask the agent to review it. Confirm that the suggested test fails for the intended business reason and passes when the correct behavior is restored. Test both known-bad and correct changes; an agent that reports every change as unsafe is not useful.

Record detection rate on seeded regressions, false positives on correct changes, reproducibility of findings, review time and execution cost. Do not treat an agent confidence score or a single successful example as evidence of reliable detection.

4. Controls and Verification of Agent Output

4.1 Access and execution boundaries

Start with read-only source access. Do not expose production credentials, customer data or a live business database. Use synthetic fixtures and an isolated test database. Treat code comments, PR text and other retrieved material as untrusted input; they cannot authorize external calls or change system instructions.

Grant narrowly scoped tool access. Test execution should use an approved runner and fixed commands; inspect test/build changes before executing branch code. Repository tests and build scripts can execute code, so an untrusted patch must not run with deployment secrets or broad network permissions. Set time, token and execution limits.

4.2 Required evidence and human review

Every finding must identify the rule, source location, expected behavior and a reproducible check. Label output as observed, inferred or unverified. A proposed command is not a test result. The runner's recorded output, not the model's wording, establishes whether execution passed or failed.

The engineer checks test independence and relevance, protects API and persisted retry contracts, and reviews the final diff. Generated tests must not weaken assertions, remove failing cases or copy an incorrect implementation as the expected result. The agent cannot merge, deploy, modify real request decisions or bypass access checks.

4.3 Supporting test commands

From Backend/BlueHarvest.PricingExceptions: dotnet build, then dotnet test. The full suite needs Docker for PostgreSQL. From FrontEnd after dependency installation: npm.cmd run build, npm.cmd run lint and npm.cmd run test:e2e. Browser tests require the configured browser and start isolated servers. Select narrower tests first when appropriate; retain required repository checks before merge.

5. Remaining Risks and Limitations

- The agent may miss a defect or misread an assumption. Human review and deterministic tests remain necessary; passing tests do not prove complete correctness.

- Generated code and generated tests may share the same mistake. Check expected results against independently agreed business rules.

- A review limited to one process cannot establish distributed safety. PostgreSQL tests must match the concurrency claim; current in-process locking does not protect multiple API instances.

- Prompt injection, sensitive-data exposure and unsafe execution are risks even in an SDLC agent. Isolation, minimal permissions and an approved execution boundary reduce but do not eliminate them.

- Outputs may vary across runs and models. Pin review inputs, retain evidence and evaluate on a repeatable set of known changes. Agent review introduces cost and latency; it should not become an unbounded release dependency.

Adoption should begin as a supervised advisory pilot. Expand access only after measured evaluation and security review. There is no automated mortgage approval agent in the application.

Repository deliverable: this standalone proposal and illustrative workflow. Existing tests and source paths were inspected for relevance; no agent deployment, mutation experiment, test execution or application-code change was performed to create it.